

**Guillermo Bejarano Reyes**

## **Guía 3. Login, autenticación básica y protección del panel administrativo**

### **Proyecto**

Tienda online básica con HTML, CSS, JavaScript, Bootstrap, Node.js y MySQL

### **Programa**

Técnico en Programación de Software

**Semana 4 y Semana 5**

## 1. Propósito de la guía

Orientar al aprendiz en la implementación de un módulo básico de autenticación para la tienda online, permitiendo el ingreso de usuarios mediante un formulario de login y protegiendo el acceso a la página administrativa de productos.

Esta guía continúa el trabajo desarrollado en:

- Guía 1: preparación del proyecto, Bootstrap y páginas iniciales: index.html, ayuda.html y contacto.html.
- Guía 2: creación de productos.html, admin-productos.html, tabla productos, archivo productos.js y rutas CRUD para productos.

Además, responde a la continuidad planteada en la Guía 2, donde se indica que el login, la autenticación básica, el despliegue y la documentación final se desarrollarán en una guía posterior.

## 2. Relación con el diseño curricular

Esta guía se articula con la fase de Ejecución, específicamente con la actividad de proyecto orientada a construir el prototipo del software de acuerdo con el análisis de las características funcionales y de calidad. En esta fase se fortalecen competencias relacionadas con la construcción de componentes frontend, la codificación del software y la integración entre interfaz, backend y base de datos.

## 3. Herramientas de trabajo

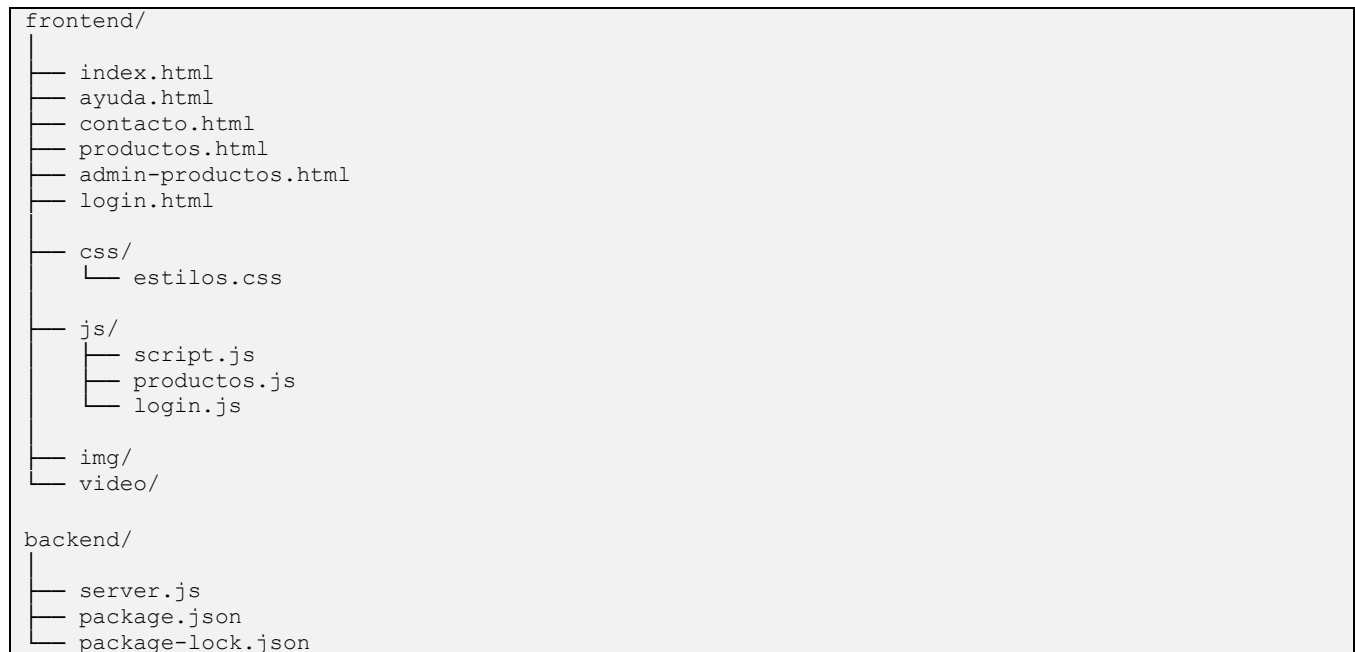
| Herramienta         | Uso                                             |
|---------------------|-------------------------------------------------|
| Visual Studio Code  | Edición del proyecto                            |
| GitHub              | Control y entrega del repositorio               |
| Live Server         | Ejecución del frontend                          |
| Bootstrap           | Diseño visual responsive                        |
| JavaScript          | Lógica del login y validaciones                 |
| Node.js             | Ejecución del backend                           |
| Express             | Creación de rutas del servidor                  |
| MySQL               | Almacenamiento de usuarios                      |
| MySQL Workbench     | Administración de la base de datos              |
| Postman o navegador | Prueba de rutas backend                         |
| Codex               | Apoyo guiado para generación y ajuste de código |

## 4. Resultado esperado de la guía

1. Página login.html creada.
2. Tabla usuarios creada en MySQL.
3. Usuario administrador registrado en la base de datos.
4. Rutas backend para login implementadas en server.js.
5. Archivo login.js creado y conectado al backend.
6. Validación básica de usuario y contraseña.
7. Protección de acceso a admin-productos.html.
8. Botón o enlace de cierre de sesión.
9. Menú de navegación actualizado.
10. Evidencias de funcionamiento.
11. Repositorio actualizado en GitHub.

## 5. Organización esperada del proyecto

El aprendiz debe conservar y ampliar la estructura del proyecto de la siguiente forma:



**Importante:** No se deben cambiar los nombres de carpetas ni eliminar archivos existentes.

## 6. Actividad 1. Revisar el estado del proyecto

### Instrucción

Antes de iniciar la autenticación, el aprendiz debe verificar que el proyecto desarrollado en las guías anteriores funcione correctamente.

1. Que index.html cargue correctamente.
2. Que ayuda.html cargue correctamente.
3. Que contacto.html conserve su formulario.
4. Que productos.html muestre el catálogo.
5. Que admin-productos.html permita registrar, listar, editar y eliminar productos.
6. Que el backend funcione con node server.js.
7. Que MySQL esté activo.
8. Que la tabla productos tenga datos.
9. Que el repositorio esté actualizado en GitHub.

```
node server.js
```

## 7. Actividad 2. Crear la tabla usuarios en MySQL

### Propósito

Crear la estructura de almacenamiento para los usuarios que podrán ingresar al panel administrativo.

## Instrucción

En MySQL Workbench, el aprendiz debe ejecutar el siguiente script:

```
USE contactos_db;

CREATE TABLE usuarios (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  correo VARCHAR(100) NOT NULL UNIQUE,
  password VARCHAR(100) NOT NULL,
  rol VARCHAR(50) DEFAULT 'admin'
);
```

## Campos de la tabla

| Campo    | Descripción                         |
|----------|-------------------------------------|
| id       | Identificador único del usuario     |
| nombre   | Nombre del usuario                  |
| correo   | Correo usado para iniciar sesión    |
| password | Contraseña del usuario              |
| rol      | Tipo de usuario, por ejemplo: admin |

## 8. Actividad 3. Insertar usuario administrador de prueba

### Propósito

Crear un usuario inicial para probar el ingreso al sistema.

### Script sugerido

```
INSERT INTO usuarios (nombre, correo, password, rol)
VALUES ('Administrador', 'admin@tienda.com', '12345', 'admin');
```

### Verificación

```
SELECT * FROM usuarios;
```

### Nota pedagógica

En esta guía se utilizará una autenticación básica para fines académicos. En proyectos reales, las contraseñas no deben guardarse en texto plano; deben cifrarse utilizando mecanismos seguros como bcrypt.

## 9. Actividad 4. Crear la página login.html

### Propósito

Crear una página que permita al usuario administrador ingresar al sistema.

### Elementos mínimos

1. Estructura HTML5 correcta.
2. Bootstrap 5 mediante CDN.
3. Barra de navegación con enlaces a: Inicio, Productos, Ayuda, Contacto y Login.
4. Título principal: Ingreso administrativo.
5. Formulario con correo electrónico, contraseña y botón ingresar.

6. Contenedor para mostrar mensajes al usuario.
7. Enlace al archivo js/login.js.
8. Pie de página básico.

### Identificadores sugeridos

```
<form id="formLogin">
<input type="email" id="correo">
<input type="password" id="password">
<div id="mensajeLogin"></div>
```

## 10. Prompt sugerido para Codex: crear login.html

Actúa como asistente de programación para un aprendiz del SENA.

Estoy trabajando en una tienda online básica con HTML, CSS, JavaScript, Bootstrap, Node.js, Express y MySQL.

Lee primero el archivo AGENTS.md y respeta sus instrucciones.

Crea el archivo frontend/login.html para permitir el ingreso administrativo a la tienda online.

Reglas:

- No cambies la estructura del proyecto.
- No modifiques backend.
- No modifiques productos.html.
- No modifiques admin-productos.html.
- No uses React, Angular ni Vue.
- Usa Bootstrap 5 mediante CDN.
- El diseño debe ser sencillo y adecuado para aprendices principiantes.
- El formulario debe tener los id: formLogin, correo, password y mensajeLogin.
- Debe enlazar el archivo js/login.js.
- Explica brevemente qué cambios realizaste.
- Indica cómo probar la página con Live Server.

La página login.html debe incluir:

1. Estructura HTML5 correcta.
2. Barra de navegación con enlaces a Inicio, Productos, Ayuda, Contacto y Login.
3. Título principal: Ingreso administrativo.
4. Formulario con correo y contraseña.
5. Botón Ingresar.
6. Mensaje de respuesta al usuario.
7. Diseño responsive con Bootstrap.
8. Pie de página básico.

## 11. Actividad 5. Crear la ruta backend para login

### Propósito

Permitir que el backend reciba los datos del formulario de login y valide si el usuario existe en la base de datos.

### Archivo a modificar

```
backend/server.js
```

### Ruta requerida

| Método | Ruta   | Función                      |
|--------|--------|------------------------------|
| POST   | /login | Validar usuario y contraseña |

## Código sugerido

```
app.post("/login", (req, res) => {
  const { correo, password } = req.body;

  if (!correo || !password) {
    return res.status(400).json({
      ok: false,
      mensaje: "Correo y contraseña son obligatorios"
    });
  }

  const sql = "SELECT id, nombre, correo, rol FROM usuarios WHERE correo = ? AND password = ?";

  db.query(sql, [correo, password], (err, resultados) => {
    if (err) {
      console.error("Error SQL:", err);
      return res.status(500).json({
        ok: false,
        mensaje: "Error en el servidor"
      });
    }

    if (resultados.length === 0) {
      return res.status(401).json({
        ok: false,
        mensaje: "Credenciales incorrectas"
      });
    }

    res.json({
      ok: true,
      mensaje: "Ingreso correcto",
      usuario: resultados[0]
    });
  });
});
```

## 12. Prompt sugerido para Codex: ruta de login

Actúa como asistente de programación para un aprendiz del SENA.

Estoy trabajando en una tienda online básica con HTML, CSS, JavaScript, Bootstrap, Node.js, Express y MySQL.

Lee primero el archivo AGENTS.md y respeta sus instrucciones.

Modifica únicamente el archivo backend/server.js para agregar una ruta POST /login.

Reglas:

- No elimines las rutas existentes.
- No elimines la ruta /guardar.
- No elimines las rutas CRUD de productos.
- No cambies la conexión actual con MySQL.
- No uses librerías adicionales.
- No uses JWT todavía.
- No uses bcrypt todavía.
- La autenticación será básica para fines académicos.
- Explica brevemente qué cambios realizaste.
- Indica cómo probar la ruta con Postman.

La ruta POST /login debe:

1. Recibir correo y password desde req.body.
2. Validar que los campos no estén vacíos.
3. Buscar el usuario en la tabla usuarios.
4. Responder con JSON.

5. Enviar ok: true si el usuario existe.
6. Enviar ok: false si las credenciales son incorrectas.

## 13. Actividad 6. Crear el archivo login.js

### Propósito

Conectar el formulario de login con el backend mediante fetch().

### Archivo a crear

frontend/js/login.js

### Código sugerido

```
const formLogin = document.getElementById("formLogin");
const mensajeLogin = document.getElementById("mensajeLogin");

formLogin.addEventListener("submit", async function (event) {
  event.preventDefault();

  const correo = document.getElementById("correo").value;
  const password = document.getElementById("password").value;

  try {
    const respuesta = await fetch("http://localhost:3000/login", {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify({ correo, password })
    });

    const datos = await respuesta.json();

    if (datos.ok) {
      localStorage.setItem("usuario", JSON.stringify(datos.usuario));
      mensajeLogin.innerHTML = `<div class="alert alert-success">${datos.mensaje}</div>`;

      setTimeout(() => {
        window.location.href = "admin-productos.html";
      }, 1000);
    } else {
      mensajeLogin.innerHTML = `<div class="alert alert-danger">${datos.mensaje}</div>`;
    }
  } catch (error) {
    console.error("Error:", error);
    mensajeLogin.innerHTML = `<div class="alert alert-danger">No se pudo conectar con el
servidor</div>`;
  }
});
```

## 14. Actividad 7. Proteger admin-productos.html

### Propósito

Evitar que un usuario ingrese directamente a la página administrativa sin haber iniciado sesión.

## Instrucción

En el archivo productos.js o en un nuevo archivo llamado auth.js, se debe verificar si existe un usuario en localStorage.

## Código sugerido

```
const usuario = localStorage.getItem("usuario");

if (!usuario) {
  alert("Debe iniciar sesión para acceder al panel administrativo");
  window.location.href = "login.html";
}
```

## Ubicación sugerida

Este código debe ejecutarse al cargar admin-productos.html.

## 15. Actividad 8. Crear cierre de sesión

### Propósito

Permitir que el usuario administrador salga del sistema.

### Botón sugerido en admin-productos.html

```
<button id="btnCerrarSesion" class="btn btn-danger">Cerrar sesión</button>
```

## Código sugerido

```
const btnCerrarSesion = document.getElementById("btnCerrarSesion");

if (btnCerrarSesion) {
  btnCerrarSesion.addEventListener("click", function () {
    localStorage.removeItem("usuario");
    window.location.href = "login.html";
  });
}
```

## 16. Actividad 9. Actualizar el menú de navegación

### Instrucción

El aprendiz debe actualizar el menú en las páginas principales:

1. index.html
2. ayuda.html
3. contacto.html
4. productos.html
5. login.html
6. admin-productos.html

### Menú sugerido

| Enlace    | Archivo        |
|-----------|----------------|
| Inicio    | index.html     |
| Productos | productos.html |



|                       |                      |
|-----------------------|----------------------|
| Ayuda                 | ayuda.html           |
| Contacto              | contacto.html        |
| Login                 | login.html           |
| Administrar productos | admin-productos.html |

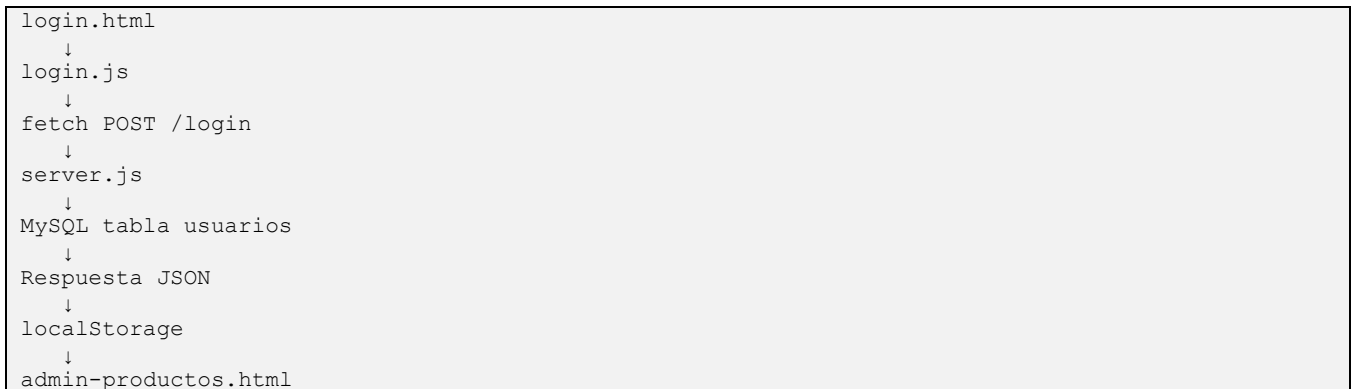
## Nota

El enlace Administrar productos debe usarse para ingresar al panel administrativo, pero si el usuario no ha iniciado sesión, debe ser redirigido a login.html.

## 17. Actividad 10. Pruebas mínimas de funcionamiento

| Prueba                                        | Resultado esperado                       |
|-----------------------------------------------|------------------------------------------|
| Abrir login.html                              | Carga el formulario de ingreso           |
| Ingresar datos correctos                      | Redirige a admin-productos.html          |
| Ingresar datos incorrectos                    | Muestra mensaje de error                 |
| Abrir admin-productos.html sin login          | Redirige a login.html                    |
| Cerrar sesión                                 | Elimina la sesión y vuelve al login      |
| Ejecutar <code>SELECT * FROM usuarios;</code> | Muestra los usuarios registrados         |
| Probar /login en Postman                      | Devuelve respuesta JSON                  |
| Ver catálogo                                  | productos.html sigue mostrando productos |
| Administrar productos                         | El CRUD sigue funcionando correctamente  |

## 18. Explicación del flujo de autenticación



## Explicación

El usuario escribe su correo y contraseña en login.html. El archivo login.js envía esos datos al backend mediante fetch(). El archivo server.js recibe la información y consulta la tabla usuarios en MySQL. Si los datos son correctos, el sistema guarda temporalmente la información del usuario en localStorage y permite ingresar a admin-productos.html. Si los datos son incorrectos, se muestra un mensaje de error.

## 19. Recomendaciones finales

1. Ejecutar primero MySQL.
2. Ejecutar después el backend con `node server.js`.
3. Abrir el frontend con Live Server.
4. Revisar la consola del navegador si ocurre un error.
5. Revisar la terminal del backend si la ruta /login falla.

6. No eliminar las rutas del CRUD de productos.
7. No modificar los nombres de archivos ya trabajados.
8. Actualizar el repositorio en GitHub al finalizar.

```
node server.js
```

## 20. Producto final de la Guía 3

Al finalizar, el aprendiz tendrá una tienda online básica con:

- Páginas informativas.
- Catálogo de productos.
- CRUD de productos.
- Base de datos MySQL.
- Backend con Express.
- Login administrativo.
- Protección básica del panel de administración.
- Repositorio actualizado con evidencias.